Лабораторная работа № 2

Тема: Основы SQL и CRUD-операции в Express.js и PostgreSQL

Цель: Реализовать полный CRUD для товаров через pg с разделением Route → Controller → Service → PostgreSQL; самостоятельно спроектировать категории и реализовать CRUD категорий.

Лабораторная работа № 2

Тема: Основы SQL и CRUD-операции в Express.js и PostgreSQL

Цель: реализовать полный CRUD для товаров интернет-магазина через pg, используя разделение Route → Controller → Service → PostgreSQL, и закрепить работу с SELECT, INSERT, UPDATE, DELETE, WHERE, RETURNING и параметризованными запросами.

1. Исходные условия и требования

Продолжайте учебный проект shop-api. Используются Node.js с ES Modules, Express 5, PostgreSQL и пакет pg. Предполагается, что сервер PostgreSQL доступен, а база shop_db и пользователь shop_user уже созданы.

Основная часть лабораторной повторяет минимальный код лекции: таблица products содержит текстовое поле category. Если после предыдущей лабораторной работа выполняется с нормализованной схемой products.category_id → categories.id, необходимо согласовать SQL-запросы с фактической схемой: примеры с полем products.category в такой БД напрямую не выполнятся. Не удаляйте существующие таблицы ради выполнения примеров.

- Основная часть: реализовать GET /api/products, GET /api/products/:id, POST /api/products, PATCH /api/products/:id, DELETE /api/products/:id.
- Самостоятельная дополнительная часть: реализовать CRUD для категорий с отдельными файлами Service, Controller и Router — без готового кода в инструкции.
- SQL-запросы к таблицам должны находиться только в Service; Controller отвечает за HTTP-ответы, Router — за маршруты.

2. Подготовьте таблицу products

При использовании новой учебной базы без таблицы products создайте таблицу. Если таблица уже существует, проверьте её структуру и не запускайте CREATE TABLE повторно.

```
CREATE TABLE products (
    id INTEGER GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
    name VARCHAR(150) NOT NULL,
    price INTEGER NOT NULL CHECK (price > 0),
    category VARCHAR(100) NOT NULL,
    in_stock BOOLEAN NOT NULL DEFAULT true
);
```

Добавьте тестовые строки только при необходимости:

```
INSERT INTO products (name, price, category, in_stock)
VALUES
    ('Keyboard', 15000, 'Accessories', true),
    ('Mouse', 8000, 'Accessories', true),
    ('Monitor', 85000, 'Monitors', false);
```

```
SELECT id, name, price, category, in_stock FROM products ORDER BY id;
```

3. Создайте структуру проекта

```
shop-api/
├── config/
│   └── db.js
├── controllers/
│   └── productController.js
├── routes/
│   └── productRoutes.js
├── services/
│   └── productService.js
├── .env
├── app.js
└── package.json
```

```
npm init -y
npm install express pg
npm pkg set type=module
npm pkg set scripts.start="node app.js"
```

Нужна версия Node.js, поддерживающая process.loadEnvFile() (например, Node.js 22). Для реального проекта добавьте .env и node_modules/ в .gitignore. Не публикуйте пароль в репозитории.

4. Настройте переменные окружения и Pool

Файл .env (пароль только для локальной учебной среды):

```
PGHOST=localhost
PGPORT=5432
PGDATABASE=shop_db
PGUSER=shop_user
PGPASSWORD=college123
```

Файл config/db.js:

```
import { loadEnvFile } from "node:process";
import pg from "pg";

loadEnvFile();

export const pool = new pg.Pool();
```

Pool использует стандартные PG-переменные окружения для соединения с PostgreSQL.

5. Реализуйте Service — SQL-запросы

Создайте services/productService.js. Обратите внимание: INSERT, UPDATE и DELETE используют RETURNING, а значения передаются отдельно от SQL через $1, $2 и далее.

```
import { pool } from "../config/db.js";

// READ — все товары
export async function getAllProducts() {
    const result = await pool.query(
        "SELECT * FROM products ORDER BY id"
    );
    return result.rows;
}

// READ — товар по id
export async function findProductById(id) {
    const result = await pool.query(
        "SELECT * FROM products WHERE id = $1",
        [id]
    );
    return result.rows[0];
}

// CREATE — новый товар
export async function addProduct(data) {
    const result = await pool.query(
        `INSERT INTO products (name, price, category, in_stock)
         VALUES ($1, $2, $3, $4)
         RETURNING *`,
        [data.name, data.price, data.category, data.inStock]
    );
    return result.rows[0];
}

// UPDATE — изменение цены
export async function updateProductById(id, price) {
    const result = await pool.query(
        `UPDATE products SET price = $1
         WHERE id = $2 RETURNING *`,
        [price, id]
    );
    return result.rows[0];
}

// DELETE — удаление товара
export async function deleteProductById(id) {
    const result = await pool.query(
        "DELETE FROM products WHERE id = $1 RETURNING *",
        [id]
    );
    return result.rows[0];
}
```

Для простоты PATCH в этой работе обновляет только price. Полное частичное обновление нескольких полей с COALESCE рассматривается в лекции и может быть выполнено отдельно.

6. Реализуйте Controller — HTTP-обработчики

Создайте controllers/productController.js. Каждый обработчик вызывает Service, возвращает JSON и отвечает 404, если товар не найден.

```
import * as service from "../services/productService.js";

export async function getProducts(req, res) {
```

```
    res.json(await service.getAllProducts());
}

export async function getProductById(req, res) {
    const product = await service.findProductById(req.params.id);
    if (!product) return res.sendStatus(404);
    res.json(product);
}

export async function createProduct(req, res) {
    const product = await service.addProduct(req.body);
    res.status(201).json(product);
}

export async function updateProduct(req, res) {
    const product = await service.updateProductById(
        req.params.id, req.body.price
    );
    if (!product) return res.sendStatus(404);
    res.json(product);
}

export async function deleteProduct(req, res) {
    const product = await service.deleteProductById(req.params.id);
    if (!product) return res.sendStatus(404);
    res.json(product);
}
```

Код предполагает Express 5, который передаёт ошибки отклонённых Promise в обработчик ошибок. В промышленном API потребуются проверка параметров, тела запроса и разграничение ошибок клиента и сервера.

7. Создайте Router

Создайте routes/productRoutes.js:

```
import { Router } from "express";
import * as controller from "../controllers/productController.js";

const router = Router();

router.get("/", controller.getProducts);
router.get("/:id", controller.getProductById);
router.post("/", controller.createProduct);
router.patch("/:id", controller.updateProduct);
router.delete("/:id", controller.deleteProduct);

export default router;
```

8. Подключите маршруты в app.js

```
import express from "express";
import productRoutes from "./routes/productRoutes.js";

const app = express();
app.use(express.json());
app.use("/api/products", productRoutes);
```

```
app.use((err, req, res, next) => {
    console.error(err);
    res.status(500).json({ message: "Server error" });
});

app.listen(3000, () => {
    console.log("Server started on port 3000");
});
```

Запустите сервер командой npm start. Если используется существующее приложение с централизованной обработкой ошибок и валидацией, сохраните его middleware вместо замены всего app.js.

9. Проверьте CRUD запросами HTTP-клиента

1. GET http://localhost:3000/api/products — HTTP 200 и массив товаров.
2. GET http://localhost:3000/api/products/1 — HTTP 200 и один товар, если id существует.
3. POST http://localhost:3000/api/products — HTTP 201 и созданный товар.
4. PATCH http://localhost:3000/api/products/1 — HTTP 200 и товар с обновлённой ценой, если id существует.
5. DELETE http://localhost:3000/api/products/1 — HTTP 200 и удалённый товар, если id существует.
6. GET с отсутствующим id — HTTP 404.

JSON-тело POST:

```
{
  "name": "SSD 2TB",
  "price": 65000,
  "category": "Components",
  "inStock": true
}
```

JSON-тело PATCH (замените id на id созданного товара):

```
{
  "price": 60000
}
```

Примечание: этот минимальный пример возвращает поле in_stock, а не inStock, потому что используется RETURNING *и SELECT* . Для сохранения старого контракта API вместо * перечислите нужные столбцы и укажите in_stock AS "inStock".

После POST проверьте SELECT в pgAdmin/psql, затем обновите товар, удалите его и убедитесь, что повторный GET по удалённому id возвращает 404. Проверьте сохранность остальных записей после перезапуска Node.js.

10. Самостоятельная работа: таблица Categories и API категорий

Эта часть выполняется студентом самостоятельно, без готового SQL и JavaScript. Она не должна мешать проверке основного CRUD товаров.

10.1. Спроектируйте таблицу categories

- Создайте таблицу categories с полями id и name.
- id — целочисленный первичный ключ с автоматической генерацией значения.

- name — строка длиной до 100 символов; значение обязательно и уникально.
- Добавьте не менее трёх категорий (например, Accessories, Monitors, Components).
- Проверьте результат командой SELECT с сортировкой по id.

Для дополнительного усложнения свяжите таблицу products с categories внешним ключом category_id и адаптируйте Product Service под эту схему. Это отдельное расширение: не добавляйте внешний ключ, не изменив SQL-запросы товаров.

10.2. Реализуйте CRUD категорий

Создайте файлы services/categoryService.js, controllers/categoryController.js, routes/categoryRoutes.js и подключите маршрутизатор в app.js. Самостоятельно определите названия функций, составьте SQL и организуйте обработку запросов.

- GET /api/categories — вернуть все категории (SELECT, ORDER BY id).
- GET /api/categories/:id — вернуть категорию по id (SELECT с WHERE); при отсутствии — 404.
- POST /api/categories — создать категорию по JSON-телу {"name":"Laptops"} (INSERT ... RETURNING), статус 201.
- PATCH /api/categories/:id — изменить название категории по JSON-телу {"name":"Displays"} (UPDATE ... WHERE ... RETURNING).
- DELETE /api/categories/:id — удалить категорию по id (DELETE ... WHERE ... RETURNING).
- Все пользовательские значения передавать через параметры $1, $2 и далее.
- Обработать ошибку повторяющегося name и учитывать связи с products, если настроен внешний ключ.

Пример ожидаемого ответа GET /api/categories (id зависят от реальных записей):

```
[
  { "id": 1, "name": "Accessories" },
  { "id": 2, "name": "Monitors" },
  { "id": 3, "name": "Components" }
]
```

Готовая реализация SQL для categories, а также коды Service, Controller и Router в лабораторной не приводятся: это проверяемая самостоятельная часть.

11. Требования к результату

- Приложение запускается из shop-api командой npm start.
- Основные пять маршрутов товаров работают с PostgreSQL, а не с массивом.
- SQL расположен в productService.js, маршрутизация и HTTP-ответы отделены от Service.
- POST возвращает 201; операции над несуществующим товаром — 404.
- SQL использует параметры вместо конкатенации пользовательского ввода.
- Основные изменения видны в PostgreSQL и сохраняются после перезапуска Node.js.
- Самостоятельно выполнены проектирование categories и пять HTTP-операций над категориями.
- В отчёт приложены структура проекта, SQL создания категорий, важные фрагменты кода, примеры HTTP-запросов и ответы, результаты проверки в PostgreSQL.

12. Контрольные вопросы

7. Какие SQL-команды соответствуют CRUD?
8. Зачем использовать WHERE в UPDATE и DELETE?
9. Что возвращают result.rows и result.rows[0]?
10. Для чего применяется RETURNING?

11. Почему параметры $1, $2 безопаснее конкатенации SQL?
12. Почему SQL-запросы размещаются в Service, а не в Controller?
13. Чем отличается статус 201 от 200?
14. Как должен вести себя GET для отсутствующего id?
15. Как ограничение UNIQUE влияет на добавление категории?
16. Что изменится в запросах products при переходе от category к category_id?